

## Week 8: Modern Asymmetric Ciphers

---

PhD. Ngoc-Tu Nguyen

[tunn@uit.edu.vn](mailto:tunn@uit.edu.vn)

# Outline

---

- Why asymmetric cryptography?
- Factoring Based Cryptography (P1)
- Logarithm Based Cryptography (P2)
  - ElGamal cipher;
  - Diffie-Hellman key exchange;
- Elliptic Curve Cryptography (P3)
- Some advanced cryptography system (quantum resistance)

# Why Public-Key Cryptosystems?

---

To overcome two of the most difficult problems associated with symmetric encryption:

- **Key distribution (key for symmetric encryption)**
  - How to have secure communications in general without having to trust a KDC with your key
- **Digital signatures**
  - How to verify that a message comes intact from the claimed sender

**Whitfield Diffie and Martin Hellman:** proposed a method that addressed both problems (1976)

# Modern Asymmetric ciphers

## Symmetric cipher vs Asymmetric cipher

Homomorphic, Searchable encryption,...

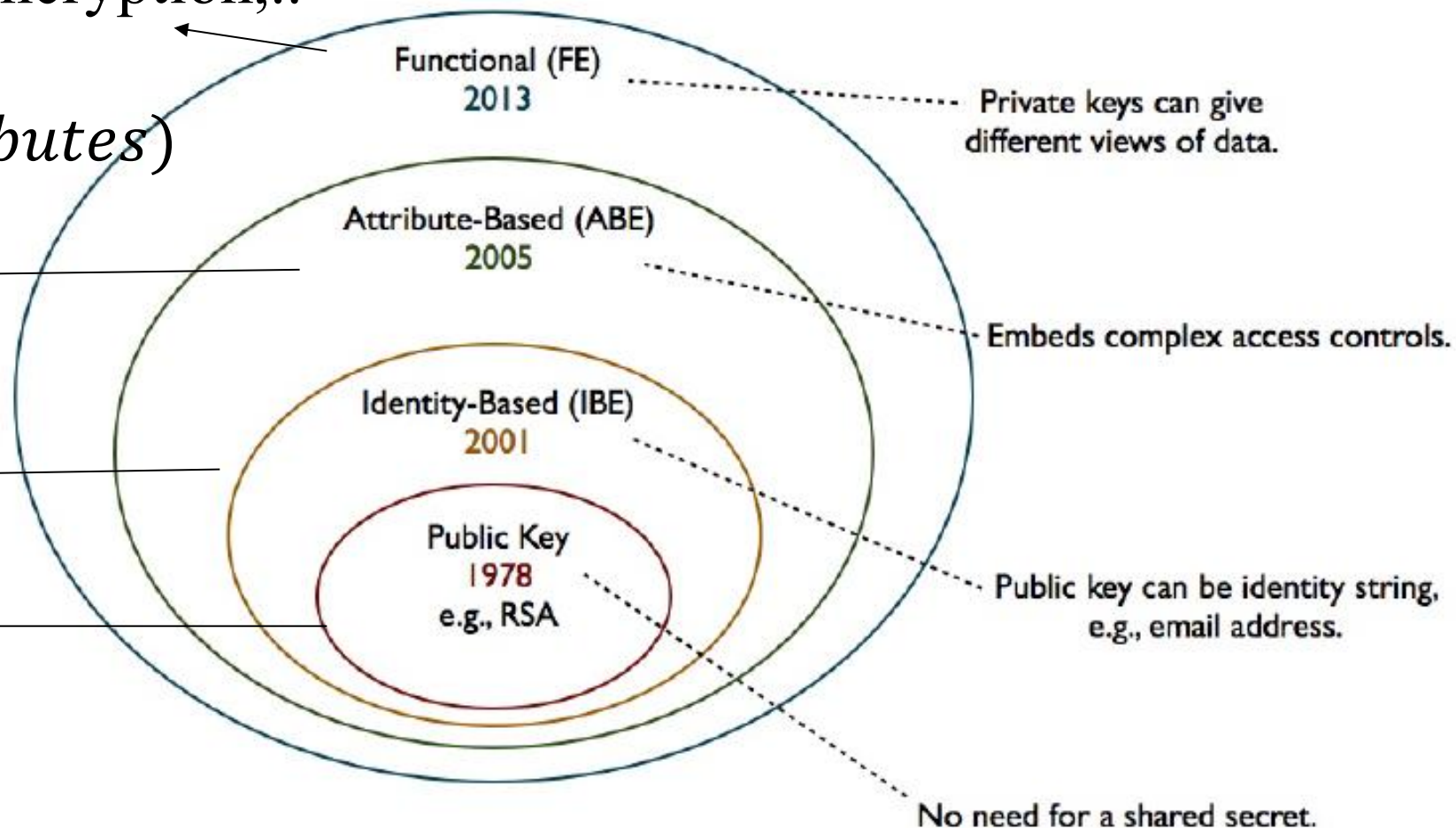
$ID_A(attributes)$   $ID_B(attributes)$

$(PK_A, \{SK_A, SK_B, \dots\})$

ID

$(PK_A, SK_A)$

$(PK_A, SK_A)$



# RSA: Confidentiality



**A**

$d_A / n_A, e_A$

$$e_A \cdot d_A = 1 \bmod \phi(n_A)$$

$$C = m^{e_B} \bmod n_B$$



$$\begin{aligned} C^{d_B} \bmod n_B &= (m^{e_B})^{d_B} \bmod n_B \\ &= m \end{aligned}$$

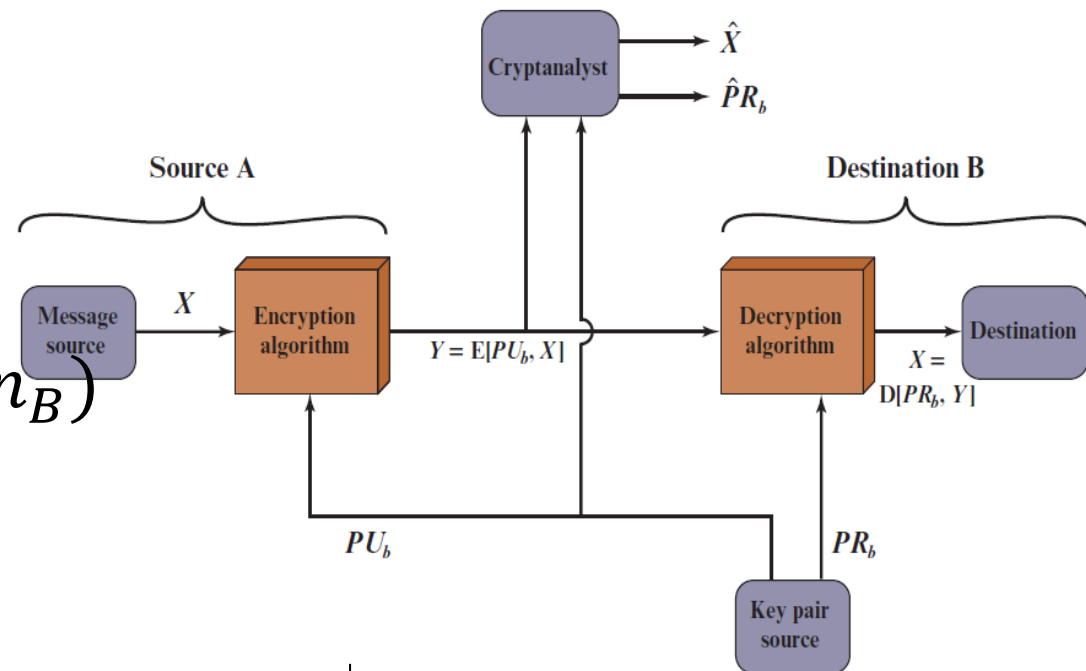
- Protect secret key?
- Distribute public keys?



**B**

$d_B / n_B, e_B$

$$e_B \cdot d_B = 1 \bmod \phi(n_B)$$



# RSA: Authentication



A

$d_A / n_A, e_A$

$$S = m^{d_A} \bmod n_A \quad (m, S)$$

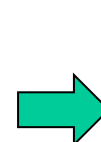


Verify message  $m$

$$S^{e_A} \bmod n_A = (m^{d_A})^{e_A} \bmod n_A$$

$$= m'$$

$$? = m$$



• Sent by A

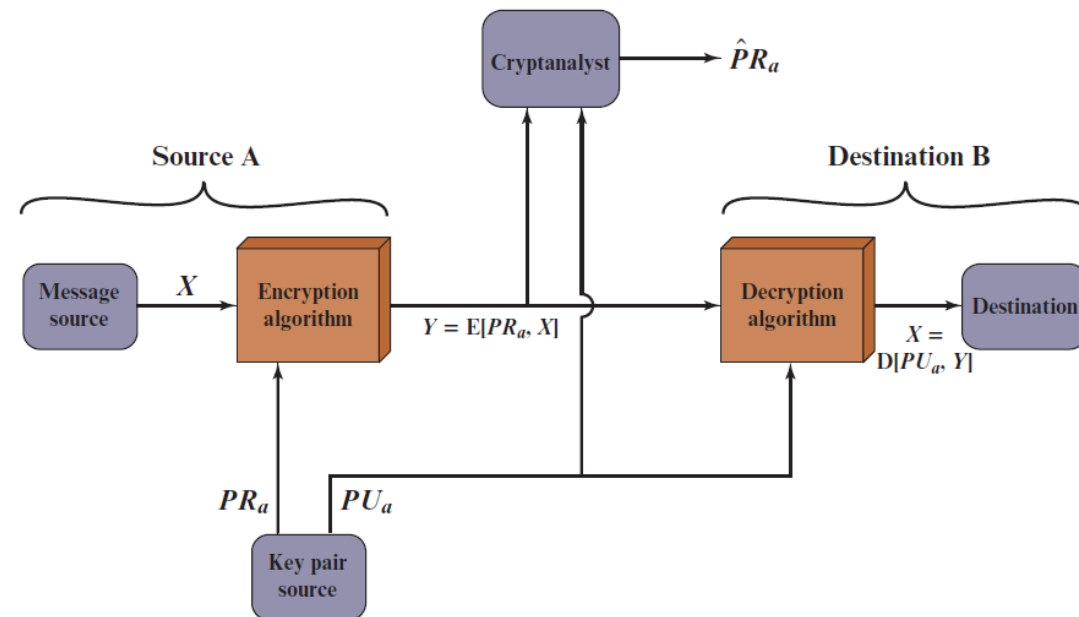
• Original (integrity)

- Protect secret key?
- Distribute public keys?



B

$d_B / n_B, e_B$



# RSA: Authentication and Secrecy



A



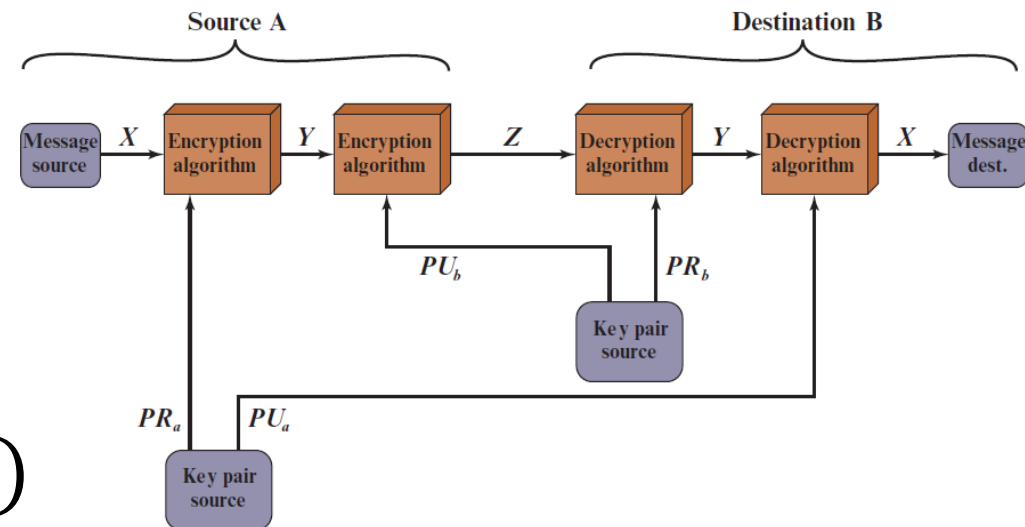
B

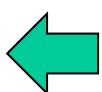
$d_A / n_A, e_A$

$d_B / n_B, e_B$

$$S = k^{d_A} \bmod n_A$$

$$(C_1 = k^{e_B}, S_1 = S^{e_B})$$



Limitation? 

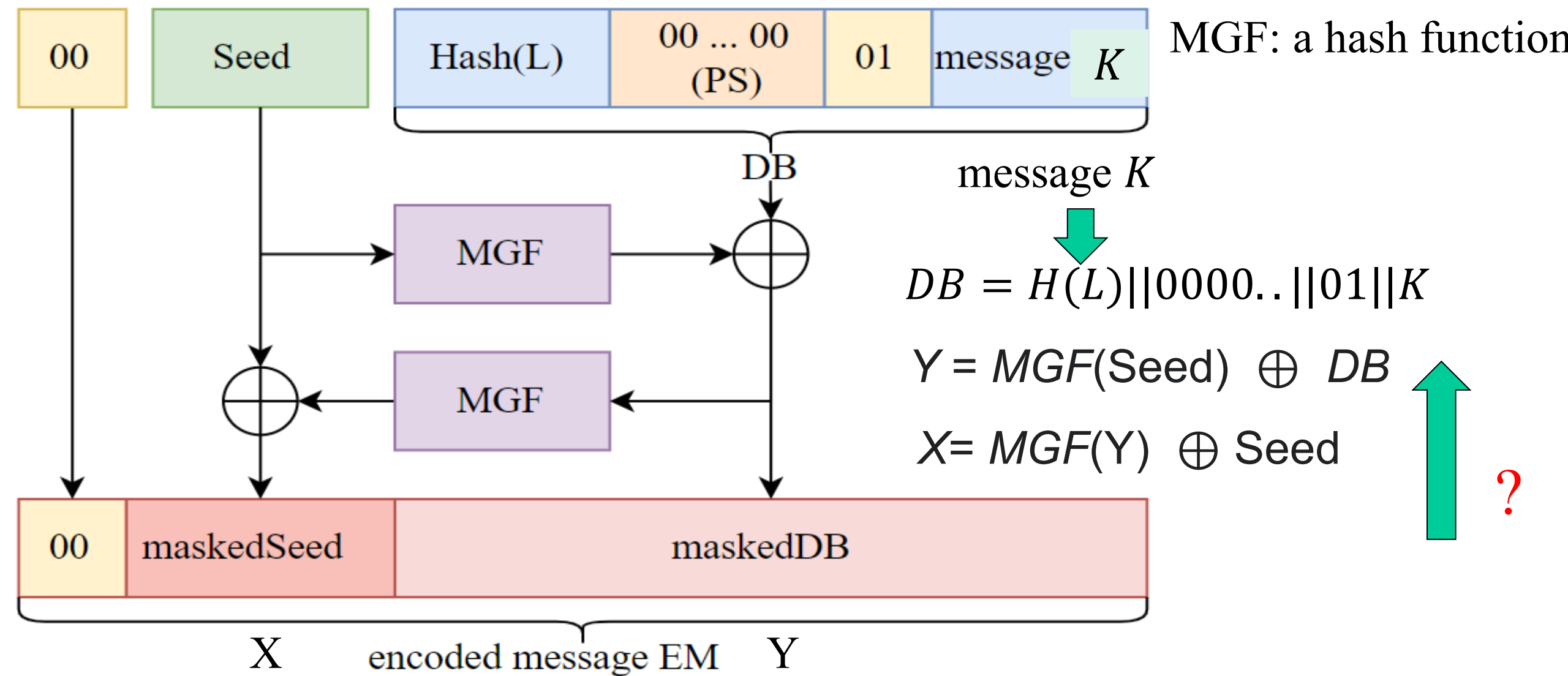
Decrypt and verify the secret key  $k$

$$C_1^{d_B} \bmod n_B = (k^{e_B})^{n_B} \bmod n_B = k;$$

$$S_1^{d_B} \bmod n_B = (S^{e_B})^{n_B} \bmod n_B = S;$$

$$S^{e_A} \bmod n_A = (k^{d_A})^{e_A} \bmod n_A = k' = ? k$$

# Encryption Using Optimal Asymmetric Encryption Padding (OAEP)



# Outline

---

- Why asymmetric cryptography?
- Factoring Based Cryptography (P1)
- Logarithm Based Cryptography (P2)
  - ElGamal cipher;
  - Diffie-Hellman key exchange;
- Elliptic Curve Cryptography (P3)
- Some advanced cryptography system (quantum resistance)

# Discrete Logarithm problem

Finite multiplicative group  $(G, \cdot) = \langle g \rangle = \{g^n : n \in \mathbb{Z}\}$

Example:  $G = \mathbb{Z}_p \setminus \{0\} = \{1, 2, \dots, p-1\} = \langle g \rangle$

$g, n$   $\xrightarrow{\text{Easy to compute}}$   $y_0 = g^n \bmod p$

$g^n = y_0 \bmod p$   $\xleftarrow{\text{Hard to solve } n}$   $g, y_0, p$

**Hard to solve equation  $g^x = a \bmod p$  in finite field!**

# Outline

---

- Why asymmetric cryptography?
- Factoring Based Cryptography (P1)
  - RSA signature;
- Logarithm Based Cryptography (P2)
  - ElGamal cipher
  - Diffie-Hellman key exchange;
- Elliptic Curve Cryptography (P3)
- Some advanced cryptography system (quantum resistance)

## ElGamal parameters

Large prime number:  $p$

Multiplicative group

$$G = \langle g \rangle = \mathbb{Z}_p \setminus \{0\} = \{1, 2, \dots, p - 1\}$$

---

## Key generation (

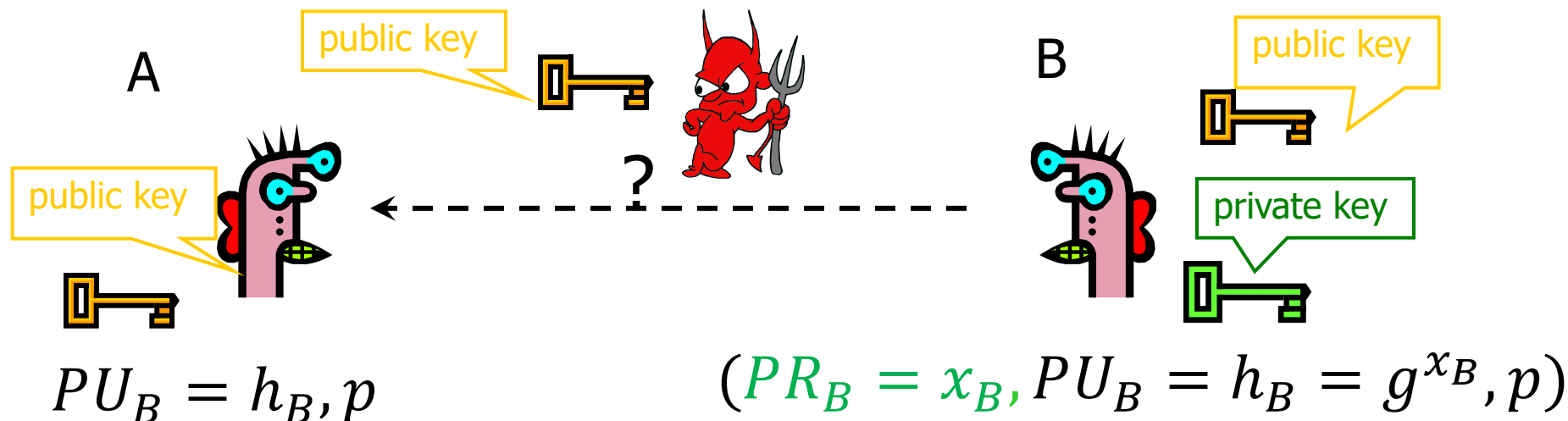
Secret key:  $x \in_R [1, p - 1]$

Public key:  $h = g^x \bmod p \in \mathbb{Z}_p$

# ElGamal cipher

- **Encryption** message  $m < p - 1$  (using public key  $h = g^x$ )
  - Choose a random number:  $r \in_R [1, p - 1]$
  - Compute  $C_1 = g^r \bmod p$ ;
  - Compute  $C_2 = m \cdot h^r \bmod p$
  - Output cipher message  $(C_1, C_2)$
- **Decryption**  $(C_1, C_2)$  (using secret key  $x$ )
  - Compute  $(C_1)^x \bmod p = g^{r \cdot x} \bmod p$ ;
  - Compute  $\frac{C_2}{(C_1)^x} \bmod p = \frac{m \cdot g^{x \cdot r}}{g^{r \cdot x}} \bmod p = m$
  - Output message  $m$

# ElGamal cipher



Input:  $M < p$

Select a random number:  $r < p - 1$

Compute:  $C_1 = g^r \bmod p$   
 $C_2 = m \cdot h_B^r \bmod p$

$(C_1, C_2)$

Compute:

$$\begin{aligned} & \frac{C_2}{(C_1)^{x_B}} \bmod p \\ &= \frac{m \cdot g^{x_B \cdot r}}{g^{r \cdot x_B}} \bmod p = m \end{aligned}$$

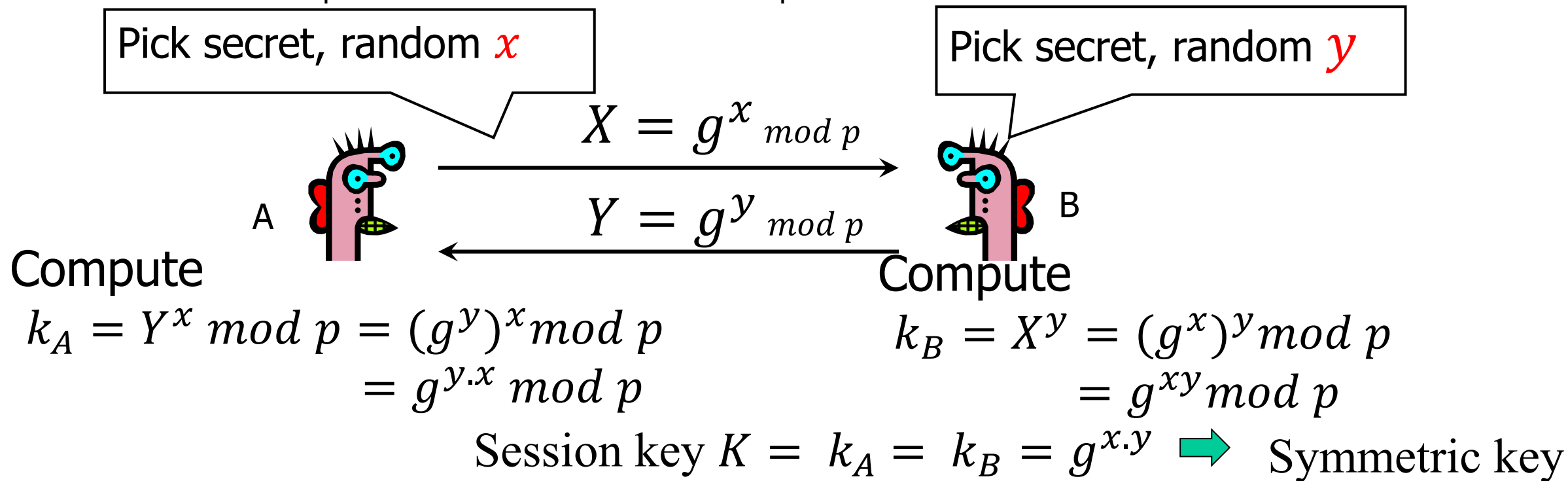
# Outline

---

- Why asymmetric cryptography?
- Factoring Based Cryptography (P1)
  - RSA signature;
- Logarithm Based Cryptography (P2)
  - ElGamal cipher
  - Diffie-Hellman key exchange;
- Elliptic Curve Cryptography (P3)
- Some advanced cryptography system (quantum resistance)

# Diffie-Hellman key exchange

- A and B never met and share no secrets;
- Public info: the prime number  $p$  and  $g$ 
  - $p$  is a large prime number,  $g$  is a generator of  $Z_p^*$ 
    - $Z_p^* = \{1, 2 \dots p-1: \forall a \in Z_p^* \exists i \text{ such that } a = g^i \bmod p\}$



# Diffie-Hellman exchange Protocol (DHE)

$p = 1606938044258990275541962092341162602522202993782792835301301$

$g = 123456789$



$g^a \bmod p =$

78467374529422653579754596319852702575499692980085777948593



$g^b \bmod p =$

560048104293218128667441021342483133802626271394299410128798

$a =$

685408003627063

761059275919665

781694368639459

527871881531452

$(g^b)^a \bmod p$

$b =$

362059131912941

987637880257325

269696682836735

524942246807440

$(g^a)^b \bmod p$

$g^{ab} \bmod p =$

437452857085801785219961443000

845969831329749878767465041215

# Why Is Diffie-Hellman Secure?

---

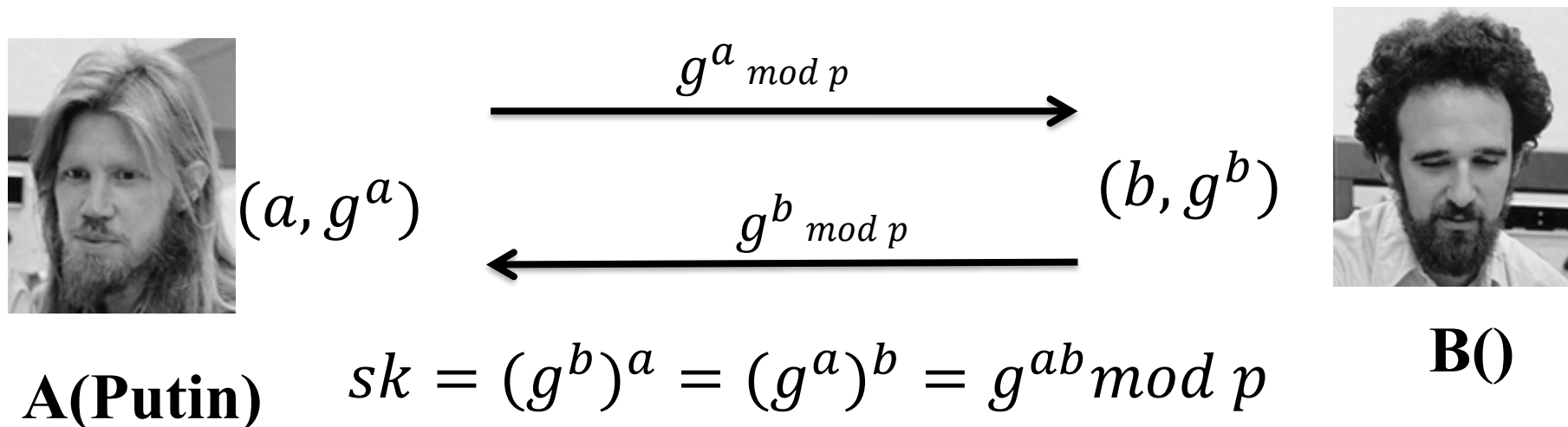
- Discrete Logarithm (DL) problem:  
given  $g^x \bmod p$ , it's hard to extract  $x$ 
  - There is no known efficient algorithm for doing this
  - This is not enough for Diffie-Hellman to be secure!
- Computational Diffie-Hellman (CDH) problem:  
given  $g^x$  and  $g^y$ , it's hard to compute  $g^{xy} \bmod p$ 
  - ... unless you know  $x$  or  $y$ , in which case it's easy
- Decisional Diffie-Hellman (DDH) problem:  
given  $g^x$  and  $g^y$ , it's hard to tell the difference between  $g^{xy} \bmod p$  and  $g^r \bmod p$  where  $r$  is random

# Properties of Diffie-Hellman

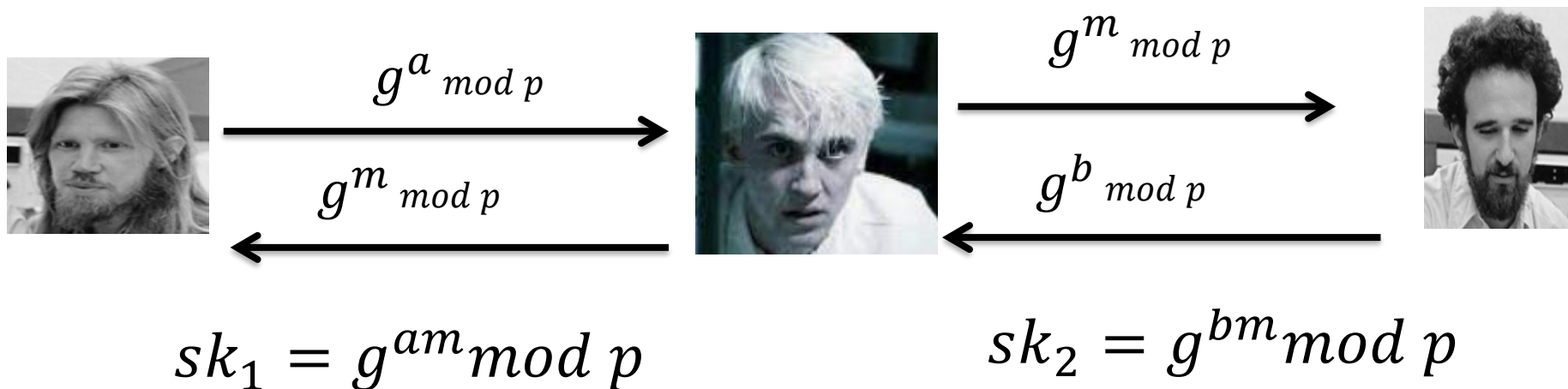
---

- Assuming DDH problem is hard, Diffie-Hellman protocol is a secure key establishment protocol against passive attackers
  - Eavesdropper can't tell the difference between the established key and a random value
  - Can use the new key for symmetric cryptography
- Basic Diffie-Hellman protocol does not provide authentication
  - IPsec combines Diffie-Hellman with signatures, anti-DoS cookies, etc.

# Man-in-the middle attacks the DHE



**man-in-the-middle attack!**



# Advantages of Public-Key Crypto

---

- Confidentiality without shared secrets
  - Very useful in open environments
  - Can use this for key establishment, avoiding the “chicken-or-egg” problem
    - With symmetric crypto, two parties must share a secret before they can exchange secret messages
- Authentication without shared secrets
- Encryption keys are public, but must be sure that Alice’s public key is really her public key
  - This is a hard problem... Often solved using public-key certificates

# Disadvantages of Public-Key Crypto

---

- Calculations are 2-3 orders of magnitude slower
  - Modular exponentiation is an expensive computation
  - Typical usage: use public-key cryptography to establish a shared secret, then switch to symmetric crypto
    - SSL, IPsec, most other systems based on public crypto
- Keys are longer
  - 3072 bits (RSA) rather than 128 bits (AES)
- Relies on unproven number-theoretic assumptions
  - Factoring, RSA problem, discrete logarithm problem, decisional Diffie-Hellman problem...